C语言指针 - 初阶与运算

理解内存地址，掌握指针的核心概念与基本运算

1. 内存

内存的概念

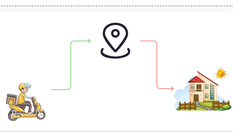
在计算机当中，所有的信息用程序都转成计成一连特殊的区域，这块区域叫做**内存**，这块**内存**被计算机划分出了不同的区域，比如栈区所熟知的，栈、堆等等。

内存很大，我们区区的变量如何标识它们在内存当中的位置？计算机给每个**位置**（通过**地址**）进行定位。

如何理解地址：

我们从外类方例，当你在了外类之后，外类小哥是如何知道你家在哪里的呢？是不是通过你填写的地址：几号楼，几单元，几室几房间定位的？

所以，也同样写到：**4号楼4单元666**，就是你在这个小区（内存）当中的位置（地址）！



2. 什么是指针变量

指针变量的定义

我们这以小的外文为例，外类小哥拿着你家的地址（**4号楼4单元666**）找到了你家，并且把外类送到你的手上，此时外类小哥就充当了指针的角色，可以认为**你存取你家的地址**，所以，指针实际上就是保存着地址的一个变量。

```
#include<stdio.h>

int main()
{
    int a = 10;
    int* p = &a;
    return 0;
}
```

- 通过 **&** 操作符可以获取变量地址
- 使用类型+星号+进行定义，可以定义指针变量

此时在p中，存储的就是变量a的地址。我们可以通过打印的方式查看地址：

```
#include<stdio.h>

int main()
{
    int a = 10;
    int* p = &a;
    printf("%d\n",&a);//或者使用%p
    printf("%d\n",p);//或者使用%p
    return 0;
}
```

3. 指针类型与访问

3.1 指针类型

定义一个char类型的指针

```
#include<stdio.h>

int main()
{
    char c = 'a';
    char* pc = &c;
    return 0;
}
```

定义一个short类型的指针

```
#include<stdio.h>

int main()
{
    short s = 10;
    short* ps = &s;
    return 0;
}
```

3.2 指针的访问

指针的间接

指针的间接指向变了指针访问的"能力"。

访问指针所指内存的操作，简称为：解引用操作，使用这算符"进行访问。

访问指针变量所指向内存的值

```
#include<stdio.h>

int main()
{
    int a = 10;
    int* p = &a;
    printf("%d\n",a);//输出变量a的值
    printf("%d\n",*p);//输出指针变量p所指向内存a的值
    return 0;
}
```

```
输出结果：
10
10
```

修改指针变量所指向内存的值

```
#include<stdio.h>

int main()
{
    int a = 10;
    int* p = &a;
    printf("%d\n",a);//输出变量a的值
    printf("%d\n",*p);//输出指针变量p所指向内存a的值

    *p = 19;
    printf("%d\n",a);//输出变量a的值
    printf("%d\n",*p);//输出指针变量p所指向内存a的值
    return 0;
}
```

```
10
通过指针修改变量内容
19
19
```

4. 指针变量的大小

指针大小的决定因素

指针变量当中存储的是地址，存储地址所需的字节数由系统架构和编译环境决定。

- **32位系统**：地址总线宽度为32位（4字节），最大寻址空间为 $2^{32} = 4GB$
- **64位系统**：地址总线宽度为64位（8字节），理论寻址空间为 2^{64} 字节（实际受操作系统限制）

观察下面代码分别在32位编译环境和64位编译环境下输出结果：

```
#include<stdio.h>

int main()
{
    printf("%d\n", sizeof(char*));
    printf("%d\n", sizeof(short*));
    printf("%d\n", sizeof(int*));
    printf("%d\n", sizeof(double*));
    return 0;
}
```

```
4
4
4
4
```

32位系统指针大小

```
8
8
8
8
```

64位系统指针大小

结论：

- 32位平台下地址是32个bit位，指针变量大小是4个字节
- 64位平台下地址是64个bit位，指针变量大小是8个字节
- **注意指针变量指向小和类型基本无关，只跟指针类型相关联，在相同的平台下，大小都是相同的。**

5. 指针运算

指针的基本运算有三种，分别是：

- 指针 + 整数
- 指针 - 指针
- 指针的关系运算

5.1 指针 + 整数

因为数组在内存中是连续存放的，只要知道第一个元素的地址，就能确定就能找到后面的所有元素。

```
int arr[10] = {1,2,3,4,5,6,7,8,9,10};
```

数组	1	2	3	4	5	6	7	8	9	10
下标	0	1	2	3	4	5	6	7	8	9

数组元素示下标

```
#include<stdio.h>
int main()
{
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    int *p = &arr[0];
    int i = 0;
    int sz = sizeof(arr)/sizeof(arr[0]);
    for(i = 0; i < sz; i++)
    {
        printf("%d ", *(p+ i));//p+i 这里就是指针+整数
    }
    return 0;
}
```

5.2 指针 - 指针

```
#include<stdio.h>
int main()
{
    int i = 0;
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    int* p1 = &arr[1];
    int* p2 = &arr[5];
    printf("%d\n",p2-p1);
    return 0;
}
```

5.3 指针的关系运算

```
//指针的关系运算：
#include<stdio.h>

int main()
{
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    int *p = &arr[0];
    int sz = sizeof(arr)/sizeof(arr[0]);
    while(p < arr + sz)//指针的大小比较
    {
        printf("%d ", *p);
        p++;
    }
    return 0;
}
```

本节总结

- 内存和地址的基本概念
- 指针变量是用于存放地址的变量
- 指针解引用操作符的间接性
- 指针大小与系统架构有关（32位为4字节，64位为8字节），与指针类型无关
- 指针可以用整数进行加减运算，用于访问数组等连续内存空间
- 两个指针相减，得到的是它们之间元素的个数
- 指针可以进行关系运算（<, >, ==等），常用于循环判断